Software Implementation and Testing Document

For

Flappy Bird Clone

Increment 2

Authors:
Andrew Bradley
Kameryn Gartrell
Luke Gilmore
Gabriel Manzo
Jake Serwe

1. Programming Languages (5 points)

Our project is a 2D, single-player desktop version of Flappy Bird. The whole game is written in C++ and is built with CMake.

Language	Where we use it	Why we chose it
C++ (C++23)	All game code: every file in `src/` and `include/`. The standard is set with `target_compile_features(prog PRIVATE cxx_std_23)` in `CMakeLists.txt`.	Everyone on the team already had C++ experience. SFML, our graphics library, is a native C++ library, so the game and the library use the same language. We looked at raylib but did not use it because it is a C library, and we did not want to put a C library inside a C++ project. C++ also gives us direct control over the real-time game loop.
CMake	`CMakeLists.txt` in the repository root.	It describes how to build the project (find SFML, list the source files, set the C++ standard) in one file. Team members use different setups (for example Ninja or MinGW Makefiles) and CMake lets all of them build the same project. Since Increment 2 it also downloads our cenui library with `FetchContent` and copies the `rsc/` folder next to the executable after every build.

Project components

This table shows where the C++ code lives and what each part is responsible for.

Component	Files	Responsibility
Entry point and main loop	`src/main.cpp`	Opens the 640x480 window and sets up a fixed 800x400 game view that is letterboxed to keep its shape when the window is resized. Loads the sounds and music, then loops once per frame: reads window events (close, resize, key press, mouse click), passes key and mouse events to the current screen, runs the physics update while the game is being played, and calls the current screen's draw function.

Game state and rules	`include/gamelogic.h`
	`src/gamelogic.cpp`
	`include/gameconfig.h`
		`GameState` holds the bird, the pipes, score, health, timers, the current screen, the player's background, pipe and bird choices, and the sounds and music. `runTickLogic` updates the game each frame (gravity, pipe movement, background scrolling, spawning and removal, scoring, damage, keeping the bird out of the floor, ceiling and pipes after a hit, and the matching sounds). `resetGame` and `playGame` start and restart a run and switch the music. `gameconfig.h` holds the tuning constants (gravity, jump speed, pipe speed and gap size, bird and pipe sizes, and the view size).

Component	Files	Responsibility
Collision	`include/collision.h` `src/collision.cpp`	Tests the bird (a circle) against the upper and lower rectangle of each pipe, and against the top and bottom of the game view.

Screens	`src/menu.cpp`
	`src/playing.cpp`
	`src/gameover.cpp`
	(and their headers)
		Each screen is a `GameStatus`: a struct of function pointers for key presses, mouse clicks and drawing. The main loop calls whichever screen is current, so adding a screen does not require changing the loop. Increment 2 used this to add the bird picker as a new screen. The menu now previews the chosen background and pipes, and lets the player change them with B and P.

Shared drawing resources	`include/gamedraw.h` `src/gamedraw.cpp`	Loads the shared font and the default bird texture, and sets up the bird sprite.
Coordinate helpers	`include/screencalc.h` `src/screencalc.cpp`	Converts between game coordinates (origin at the center of the view, y pointing up) and SFML view coordinates (origin at the top-left, y pointing down), and sets up the letterboxed view.
Backgrounds	`include/background.h` `src/background.cpp`	Loads the four background themes (Classic, Mushroom Kingdom, Minecraft, Hallownest) and draws the chosen one as a sky plus four parallax layers that scroll at different speeds and shift slightly with the bird's height.
Pipe skins	`include/pipes.h` `src/pipes.cpp`	Loads the four pipe skins (Classic, Mastersword, Lightsaber, Plant) and draws pipes of any length. Skins that should not be stretched are drawn in pieces so their ends keep their shape.
Bird picker	`include/birdselect.h` `src/birdselect.cpp`	The Bird button on the main menu and the picker screen it opens, where the player chooses one of eight birds. The choice is used on the menu and in gameplay.

C++ features we rely on

- Designated initializers (C++20) define each screen, for example `.keyPressed = keyPressedMenu, .draw = drawMenu`.
- `std::erase_if` removes pipes that have scrolled off the left edge of the window.
- `constexpr`, `noexcept` and `[[nodiscard]]` on the small collision helper functions, and `inline constexpr` for the constants in `gameconfig.h`.
- `std::array` and a function-local `static` vector that loads the bird skins the first time they are needed, and `std::fmod` to wrap the scrolling background layers.

2. Platforms, APIs, Databases, and other technologies used (5 points)

The game is one local executable. It does not use a database, a network connection or any web API while it runs (the only network access is at configure time, when CMake downloads cenui). All game data lives in memory while the game runs, and all art, fonts and audio are loaded from the `rsc/` folder when the program starts.

Technology	Details	Where we use it
SFML	Version 3 (`find_package(SFML 3 REQUIRED COMPONENTS Graphics Audio)`). 3.0.2 from MSYS2 on our Windows machines.	The Graphics module (which includes the Window and System modules), used in every component. Window and events: `sf::RenderWindow`, `sf::View` (letterboxing), `sf::Event` (close, resize, key press, mouse click). Drawing: sprites, rectangles and text. Loading: textures and fonts. Timing: `sf::Clock` and `sf::Time` for frame time, pipe spawn timing, the 1-second invulnerability window and damage flash, and the menu animations. The Audio module plays the sound effects (`sf::SoundBuffer`, `sf::Sound`) and the music (`sf::Music`).
cenui	Our own UI library (`drewand1/cenui`, v1.0.0), downloaded by CMake with `FetchContent`.	Linked into the game and included in `main.cpp` to prepare for future UI work. No screen uses it yet.
CMake	3.28 or newer	Build configuration in `CMakeLists.txt`, including downloading cenui and copying `rsc/` next to the executable after each build. The generator is up to each developer (we have used Ninja and MinGW Makefiles).
GCC (MinGW-w64) from MSYS2 UCRT64	Compiler and toolchain	Our standard compiler on Windows. We chose it over MSVC to keep everyone on one toolchain; MSVC treats some implicit 64-bit to 32-bit conversions as errors, which can break builds that GCC accepts.
Git and GitHub	Repository `drewand1/flappy-bird-CEN4090Lg2`	Version control and collaboration. Work is done on feature branches and merged through pull requests. Increment 1 used `collision`, `4-healthpoint-system`, `architectural-overhaul-1` and `increment-1`. Increment 2 used `letterbox`, `background-texture`, `background-themes`, `link-cenui`, `pipe-skins`, `bird-skins` and `sounds`.
VS Code, CMake Tools, clangd	Used by at least some of us	Editing, configuring and building the project, and static checks while typing (see Non-Execution-based Testing).
Press Start 2P font	Google Fonts, SIL Open Font License	`rsc/fonts/PressStart2P-Regular.ttf`: all in-game text (menu title and prompt, the score and lives display, and the game-over screen). `Bungee-Regular.ttf` is also in the folder as an alternative, but the code does not use it.
PNG sprites	`rsc/textures/`	Bird, pipe and background images, loaded as `sf::Texture` and drawn as `sf::Sprite`. Texture smoothing is turned off for the pixel art so it stays sharp. Backgrounds are in `backgrounds/<theme>/` (a sky and four parallax layers per theme), pipe skins are in `pipes/`, and the bird skins are in `birds/`. `ground.png` was cut from the free "16x16 tileset - Dirt and grass" pack by kanti1984 on itch.io (`dirt_grass_new.png`). `groundtiles.png` and `spikes.png` are in the folder but the code does not use them.

Technology	Details	Where we use it
WAV audio	`rsc/audio/`	Flap, hit and score sound effects, plus looping main-menu music and game-over music, played through SFML Audio.
C++ standard library		`std::vector` (pipes, loaded bird skins), `std::array` (bird skin file list), `std::clamp` and `std::erase_if`, `std::sin`, `std::ceil` and `std::fmod` (menu animation, layout and background scrolling), `std::string` (score text and asset paths), `std::optional` (returned by SFML 3 event polling), and `rand`/`srand` (random pipe gap heights).

Platform and runtime requirements

- **Target platform:** desktop. We build and run on Windows with the MinGW-w64 toolchain. SFML and CMake are cross-platform, so Linux and macOS should work, but we have not tried them.
- **SFML must be installed on the developer's machine.** The project finds it with `find_package`; earlier versions downloaded it during configure. On MSYS2 UCRT64 the command is `pacman -S mingw-w64-ucrt-x86_64-sfml`.
- **The `rsc/` folder must be in the directory the program is run from.** Assets are loaded with relative paths such as `rsc/textures/bird.png`. Since Increment 2 the build copies `rsc/` next to the executable automatically, so this no longer has to be done by hand.

3. Execution-based Functional Testing (10 points)

Approach. Functional testing for Increment 2 was again manual and execution-based. We build the game with CMake, launch the executable with the `rsc/` folder next to it, and exercise each feature with the keyboard and mouse, comparing what happens on screen with the expected behavior. We have not written automated tests yet: there is no test framework or test target in the repository. F1 to F11 cover the Increment 1 features and were run again; F12 to F20 cover the features added in Increment 2.

ID	Feature	How we test it	Expected result	Status
F1	Menu display	Launch `prog` from a folder that contains `rsc/`.	The window opens on the main menu: the chosen background scrolling slowly, two decorative pipes, a bobbing bird, the title "Flappy Bird", a pulsing "Press Space to Start", the background and pipe options, and the Bird button.	Passed
F2	Menu is idle	Leave the menu untouched for 30 seconds or more.	The game stays on the menu. No pipes appear, no life is lost and the game-over screen does not show.	Passed
F3	Start the game	Press Space on the menu.	Gameplay starts. The top-right display shows Score: 0 and Lives: 3 and the bird is on screen.	Passed

ID	Feature	How we test it	Expected result	Status
F4	Flap and gravity	During play, press a key; then press nothing.	Each key press sends the bird upward and it then falls under gravity. With no input it falls to the floor.	Passed
F5	Pipes	Play for at least 15 seconds.	A new pipe appears at the right edge about every 5 seconds with a random gap height, scrolls left at a constant speed, and is removed once it leaves the left edge.	Passed
F6	Scoring	Fly through a pipe gap.	Score goes up by 1 when the bird passes the pipe, and only once per pipe.	Passed
F7	Pipe collision	Start a run with 3 lives and fly into a pipe once.	Lives drop from 3 to 2. Exactly one life is lost per hit.	Passed
F8	Floor and ceiling	Let the bird fall to the floor without flapping. In another run, fly to the top of the window.	Each counts as a collision and costs a life, and the bird stays at the floor or ceiling instead of leaving the screen. While it stays there, a life is lost about once per second until game over.	Passed
F9	Game over	Lose all 3 lives.	A dimmed "GAME OVER" screen shows "Play Again" and "Main Menu" buttons. Everything behind it stops moving.	Passed
F10	Play again	On the game-over screen, click "Play Again".	A new run starts with Score 0, Lives 3, no old pipes and the bird reset.	Passed
F11	Back to menu	On the game-over screen, click "Main Menu".	The main menu appears. Pressing Space starts a fresh run.	Passed
F12	Background themes	On the menu, press B several times, then start a game.	The background cycles through Classic, Mushroom Kingdom, Minecraft and Hallownest and back to Classic. The label updates and gameplay uses the chosen background.	Passed
F13	Parallax scrolling	Play for about 10 seconds, flying up and down.	The background layers scroll at different speeds and shift slightly as the bird moves up and down.	Passed
F14	Pipe skins	On the menu, press P several times, then start a game.	The pipes cycle through the four skins. The menu pipes and the gameplay pipes match the choice and look correct at any length.	Passed
F15	Bird picker	Click the Bird button, choose a bird, then click Done. Repeat using Esc or Enter to close.	The picker opens, the chosen bird is outlined, and the menu and gameplay use that bird.	Passed

ID	Feature	How we test it	Expected result	Status
F16	Choices are kept	Change the background, pipes and bird, play until game over, then use Play Again and Main Menu.	All three choices stay the same across runs.	Passed
F17	Damage feedback	Fly into a pipe.	The hit sound plays, the bird flashes for about a second, and it stops at the edge of the pipe instead of passing through it.	Passed
F18	Sound effects	Flap, fly through a gap, and hit a pipe.	Each action plays its own sound.	Passed
F19	Music	Go from the menu to gameplay, to game over, and back to the menu.	Menu music loops on the menu and stops when play starts. Game-over music plays at game over, and the menu music starts again on return.	Passed
F20	Window resizing	Resize the window to a wide shape and a tall shape, on the menu and during play.	The game keeps its shape with bars on the sides or top and bottom, and the buttons still respond where they are drawn.	Passed

Results so far

- **Test run:** all 20 cases (F1 to F20) were run by hand on `main` on Windows in November 2026, and all 20 passed.
- **Defect from Increment 1, now fixed:** the bird used to fall out of view after its first floor hit and keep losing a life until game over. It is now stopped at the floor, ceiling and pipe edges on every hit, and F8 and F17 confirm it.
- **Known limitation from Increment 1, now fixed:** resizing the window could make a pipe impossible to pass. The game now uses a fixed, letterboxed view, and F20 confirms it.

4. Execution-based Non-Functional Testing (10 points)

The Requirements and Design document is where our non-functional requirements are stated. Below are the quality attributes we have checked by running or building the project, how we checked them, and what we found.

Quality attribute	How we tested it	Result
Platform / portability	Configured and built `main` from scratch on Windows with CMake and GCC from MSYS2 UCRT64.	Builds cleanly, and the game ran on Windows through all functional test cases (F1 to F20). Linux and macOS have not been tried. One risk for other platforms: the code opens `mainMenu.wav`, but the file is named `mainmenu.wav`. This works on Windows, where file names are not case-sensitive, but the menu music would not play on Linux.

Quality attribute	How we tested it	Result
Build setup	Set up a machine that had used the earlier CMakeLists (which downloaded SFML during configure). After the overhaul, configure failed with "Could not find a package configuration file provided by SFML". We installed SFML with `pacman -S mingw-w64-ucrt-x86_64-sfml`; this first needed `pacman -Syu` because the package database was out of date.	After installing SFML, configure reported "Found SFML 3.0.2" and the build finished with exit code 0. Setup works but needs one extra step per developer, which the README does not describe yet. Since Increment 2 the first configure also needs network access, because CMake downloads cenui.
Robustness: missing assets	Launched `prog.exe` from a folder that did not contain `rsc/`. This happened twice during development, first for a texture and later for a font.	The program stops immediately with an `sf::Exception` that names the missing file, for example "Failed to load image ... rsc/textures/bird.png: No such file or directory". The message is clear but there is no on-screen error. Since Increment 2, missing background or pipe images fall back to the Classic art and missing sound files are skipped, so only the font and bird images still stop the program.
Performance	VSync is turned on in `main.cpp`, so the frame rate follows the monitor's refresh rate. Each frame draws a small number of sprites (background layers, pipes and the bird), shapes and text objects.	We have not measured frame rate or CPU use with a profiler.
Display scaling (window resize)	Resized the window to wide and tall shapes on the menu and during play (F20).	The game draws into a fixed 800x400 view that is letterboxed into the window, so the layout and pipe gaps no longer depend on the window size. The Increment 1 limitation, where a pipe could become impossible to pass after a resize, is fixed.

We did not do load, security or accessibility testing, and we have not run a usability session with people outside the team. For a single-player, offline desktop game of this size we treated these as out of scope for Increments 1 and 2.

5. Non-Execution-based Testing (10 points)

We did not hold formal inspections or use a review checklist. These are the checks we did use.

- **Static analysis in the editor.** clangd (in VS Code) checks the code as it is written. It flagged a real defect in `menu.cpp` before the program was run: the line `sf::RectangleShape sky(sf::Vector2f(winSz));` was being read as a function declaration (the "most vexing parse"), so every later use of `sky` produced errors. It was fixed by using brace initialization, `sf::Vector2f{winSz}`. clangd also reported unused `#include` lines in `main.cpp` (hints only).
- **Design review before a major change.** Before the architectural overhaul (replacing the `GameStatus` enum and the `if/else` chains in `main.cpp` with a struct of function pointers per screen) was started, the plan was posted to the team's Discord with its reasoning and a request for objections or alternatives. The work was then done on its own branch, `architectural-overhaul-1`, so `main` was not changed until it was ready.

- **Pull requests.** Feature work was merged into `main` through pull requests (#13 collision, #15 health and score, #18 architectural overhaul). The pull requests themselves have no recorded reviews: #13, #15 and #18 show no reviewers and no comments from anyone but their authors. For the architectural overhaul, the discussion happened in Discord (see the previous point), and the pull request links to it. #15 was linked to its tracking issue (#4, Health/Point system). Otherwise the pull requests work as a record of what was merged rather than as a review step.
- **Pull requests in Increment 2.** All Increment 2 work (letterboxing, the parallax background, cenui, background themes, pipe skins, the bird picker and sound) was also submitted and merged into `main` through pull requests (#19, #22, #31, #32, #35, #36 and #37). None of them needed significant changes before merging.
- **Integration check after the overhaul.** Once the overhaul branch was ready, a teammate pulled it, did a clean configure and build, and confirmed that the menu code written earlier compiled and linked together with the new `playing`, `gameover` and `collision` files.
- **Advance notice for shared files.** Before pushing the menu, which touches the shared `main.cpp` and `gamelogic.h`, the author posted in Discord what would change and gave anyone editing those files a chance to speak up, to avoid conflicting edits.